Bulwark – Project Setup Guide

Oct 26, 2026 · @Srini

This guide takes a clean machine to a running Bulwark: PostgreSQL with demo data, the API and worker, both web apps, the scan ingestor, the recommender and the AI assistant. Follow the steps in order; the first three give you a working demo without Kafka or Gemini.

1. What you are installing

Six projects, one database. Start them in the order of the table; the API is the only service the UI and the AI tools talk to.

Order	Project	Port	Start command	Needs
1	PostgreSQL + pgvector	5432	(service)	consolidated DDL and inserts
2	bulwark-api	8000	`uvicorn bulwark.main:app --app-dir src --reload`	database
3	bulwark-api worker	none	`PYTHONPATH=src python -m bulwark.modules.execution.worker`	database
4	bulwark-ui	3000	`npm run dev`	API, sibling bulwark-agent-ui folder
5	bulwark-scan-ingestor	none	`scan-ingestor`	database, Kafka
6	recommender	none	`python -m recommender`	database, Kafka, Gemini key
7	bulwark-mcp	8765	`bulwark-mcp`	API

Order	Project	Port	Start command	Needs
8	bulwark-chat-server	8800	`bulwark-chat-server`	MCP server, Gemini key, database
9	bulwark-agent-ui (optional standalone)	3100	`npm run dev`	chat server

A working demo needs only rows 1 to 4. Rows 5 to 9 add scan ingestion, AI suggestions and the assistant.

Keep the six folders side by side exactly as delivered: bulwark-ui imports the chat component from `../bulwark-agent-ui`, and the API's scan module defaults to `../bulwark-scan-ingestor`.

2. Prerequisites

Tool	Version	Used by
Python	3.11 or newer (Dockerfiles use 3.12)	API, ingestor, recommender, MCP, chat agent
Node.js and npm	18 or newer (Vite 5)	both web apps
PostgreSQL	14 or newer; tested on 16	all
pgvector extension	0.5 or newer (HNSW index)	incidents.embedding, recommender, semantic search
Kafka	Aiven (SASL_SSL) or a local Redpanda	ingestor, recommender, API (optional publish)
Gemini API key	free key from Google AI Studio	embeddings, AI planner, chat agent
Docker	optional	compose files in bulwark-api, bulwark-ai, bulwark-scan-ingestor

Use a separate virtual environment per Python project:

```
python -m venv .venv && source .venv/bin/activate      # Windows:
.venv\Scripts\activate
```

3. Step 1 – Database

Create an empty database, then run the two consolidated scripts. Both were tested on PostgreSQL 16 with pgvector 0.6 and load without errors.

```
createdb bulwark                        # or create it in Supabase / RDS
export DATABASE_URL=postgresql://user:password@localhost:5432/bulwark

psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f bulwark_consolidated_ddl.sql
# 26 tables, 1 view
psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f bulwark_consolidated_inserts.sql
# demo data (optional)
```

- The DDL runs `CREATE EXTENSION IF NOT EXISTS vector`. The database user needs permission for that. On a self-hosted server install the package first (for example `postgresql-16-pgvector`); on Supabase enable the *vector* extension.
- Skip the inserts for an empty production database; run `python scripts/init_auth.py` from bulwark-api instead to create the `admin` user.
- If you later use Alembic from bulwark-api, record that the schema already exists: `alembic stamp head`. Do not run `alembic upgrade head` on top of the consolidated DDL.
- The sample data uses "now" = 23 Oct 2026, and every sample user has the default password `admin@123` and must change it at first sign-in. Do not load it into production.
- Embeddings stay empty until the recommender or ingestor fills them (step 7).

Check the load:

```
psql "$DATABASE_URL" -c "select count(*) from incidents"           # 56
psql "$DATABASE_URL" -f bulwark-api/sql/03_verify.sql               # every
check must show 0 rows (check 16 is informational)
```

4. Step 2 – bulwark-api and worker

```
cd bulwark-api
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
cp .env.example .env            # then edit it (below)

uvicorn bulwark.main:app --app-dir src --reload                  # terminal
1: http://localhost:8000/docs
PYTHONPATH=src python -m bulwark.modules.execution.worker        # terminal
2: without it runs stay Queued
```

Edit `.env` before the first start:

- `DATABASE_URL` must use the async driver: `postgresql+asyncpg://user:password@host:5432/bulwark`. If your provider offers a transaction pooler (port 6543) and you see prepared-statement errors, use the direct or session port 5432.
- `JWT_SECRET`: a long random value, not `change-me`. The chat agent needs the same value as `BULWARK_JWT_SECRET`.
- `CORS_ORIGINS`: a JSON list of the UI origins, for example `["http://localhost:3000","http://localhost:3100"]`.
- `SIMULATE_EXECUTORS=true` for demos. Leave it true until the real Ansible / SSM executors exist.
- `KAFKA_BOOTSTRAP_SERVERS` empty disables event publishing from the API.
- `GEMINI_API_KEY`, `GEMINI_EMBED_MODEL`, `EMBED_DIM=768` turn on semantic search (/insights).
- `SCAN_INBOX`, `SCAN_INGESTOR_DIR`, `SCAN_INGESTOR_CMD` let the API validate and ingest workbooks; use absolute paths with forward slashes.
- Over HTTPS set `REFRESH_COOKIE_SECURE=true`; if UI and API are on different sites also `REFRESH_COOKIE_SAMESITE=none`.

First sign-in. With the sample inserts loaded: username `admin`, password `admin@123`; the UI makes you change it at once. On a database without the sample data run `python scripts/init_auth.py` once to create `admin`. For scripts and the Swagger Authorize button: `python scripts/dev_token.py admin 60`.

Health: `GET /health/live` and `GET /health/ready` (checks the database).

5. Step 3 – Web apps

```
cd bulwark-ui
npm install
npm run dev            # http://localhost:3000
npm run build          # static files in dist/ for any web server

cd ../bulwark-agent-ui  # optional standalone chat app
npm install
npm run dev            # http://localhost:3100
```

The backend address is read in this order: the gear icon in the header (stored in the browser), then `public/config.js` (copied into `dist/`, so you can change it after a build), then `VITE_API_BASE_URL` at build time, then `http://localhost:8000`. The assistant address works the same way (`agentUrl`, default `http://localhost:8800`).

Two things commonly go wrong: the API's `CORS_ORIGINS` must contain the UI's exact origin (credentials are sent only to listed origins), and the worker must be running or executions never leave Queued.

The standalone agent UI signs in by pasting a Bulwark JWT (from `scripts/dev_token.py` or your identity provider); bulwark-ui reuses its own sign-in for the floating chat icon.

6. Step 4 – Scan ingestor and Kafka

```
cd bulwark-scan-ingestor
pip install -e ".[dev]"
cp .env.example .env     # every key is required; the app has no defaults
```

In `.env` set `DATABASE_URL` (plain `postgresql://`, psycopg), the Kafka block (`KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_SASL_USERNAME`, `KAFKA_SASL_PASSWORD`, `KAFKA_SSL_CA_FILE=./certs/ca.pem`) and `GEMINI_API_KEY` for embeddings. Put the broker's CA certificate at `certs/ca.pem`.

No Kafka yet? Use the bundled local broker: `docker compose up -d postgres redpanda` in this folder, then set `KAFKA_SECURITY_PROTOCOL=PLAINTEXT`, `KAFKA_BOOTSTRAP_SERVERS=localhost:9092`, `KAFKA_CREATE_TOPICS=true`, `KAFKA_TOPIC_REPLICATION=1`. (That compose file also loads the older ingestor schema; for the full system keep using the consolidated DDL on your own database.)

```
scan-ingestor --validate-only     # parse ./data/vulnerabilities, touch
nothing
scan-ingestor --kafka-dry-run     # real database, Kafka messages written to
reports/kafka_dry_run.jsonl
scan-ingestor                     # real run; processed files move to
processed/, rejected to failed/
scan-ingestor --watch --interval 60
```

Sample workbooks are in `samples/`; copy one into `data/vulnerabilities/`. Exit codes: 0 ok, 1 file rejected or Kafka delivery failed, 2 configuration or Kafka unreachable, 3 another instance is running.

The API can run the same job from the UI side: `POST /scans/ingest` (role ANALYST) when `SCAN_INBOX` points at this folder.

7. Step 5 – Recommender and AI planner

The recommender's tables and the pgvector column are already in the consolidated DDL (parts 2 to 4), so the three SQL files in its folder are not needed.

```
cd bulwark-inc-planner-solution-recommender
pip install -e ".[dev]"
cp .env.example .env
python -m recommender --backfill-embeddings     # embed every existing
incident once
python -m recommender                           # consume bulwark.incidents
until Ctrl+C
```

Set in `.env`: `DATABASE_URL`, the Kafka block, `GEMINI_API_KEY`, and `ENGINES=vector-similarity,ai-planner` (the planner must come second because it reads the similarity matches). Tuning knobs: `TOP_K=10`, `MIN_SIMILARITY=0.50`, `PLAN_SIMILAR_INCIDENTS=5`, `PLAN_REQUESTS_PER_MINUTE=10`.

Other modes:

- `python -m recommender --incident 57` finds similar incidents and an AI plan for INC-57 now, without Kafka. This is the quickest way to see the feature.
- `python -m recommender --once` exits after 15 seconds without messages (cron style).
- `python -m recommender --from-file events.jsonl` replays the file written by the ingestor's dry run.

A fresh database has no earlier approved plans, so results are `NO_MATCH` until some incidents have plans. The sample data provides 49 incidents with plans, so matches appear after the backfill. Calibrate `MIN_SIMILARITY` by looking at `v_incident_similar_matches.similarity` for pairs you know are alike and unlike.

8. Step 6 – MCP server and chat agent

The MCP server needs only the API address. The chat agent needs the MCP server, a Gemini key and, for saved chats, the database.

```
cd bulwark-ai/bulwark-mcp-server
cp .env.example .env                # BULWARK_API_URL=http://localhost:8000
pip install -e ".[dev]"
bulwark-mcp                         # http://127.0.0.1:8765/mcp   (health:
/health)

cd ../bulwark-chat-agent
cp .env.example .env                # GEMINI_API_KEY, BULWARK_JWT_SECRET (=
API JWT_SECRET), MCP_URL, CHAT_DATABASE_URL
pip install -e ".[dev]"
bulwark-chat-server                 # HTTP API on :8800 (the web chat talks
to this)
bulwark-chat --user ravi --roles ANALYST,OPERATOR     # or a terminal chat as
a development user
```

Notes:

- Chat history lives in schema `bulwark_chat`, already created by part 5 of the consolidated DDL. Set `CHAT_DATABASE_URL` to the direct or session-pooler port 5432, not a transaction pooler. Without it history is kept in memory only.
- `BULWARK_JWT_SECRET` is for local development token minting and for verifying who owns a chat; in production send each user's real JWT.
- `REQUIRE_CONFIRMATION=true` (default) makes every data-changing tool wait for a human yes. Do not switch it off in shared environments.
- `CORS_ORIGINS` on the chat server must include `http://localhost:3000` and `http://localhost:3100`.
- On Windows the chat server must reach the MCP server over HTTP (the default), not stdio.

- Any MCP client can use the tools. For Claude Desktop use the stdio command `python -m bulwark_mcp --transport stdio` with `BULWARK_API_URL` and `BULWARK_API_TOKEN` in its environment.

9. Configuration reference

Which values must agree across projects, and which are specific to one.

Setting	Where	Rule
DATABASE_URL	API	`postgresql+asyncpg://…` (async driver)
DATABASE_URL	ingestor, recommender	plain `postgresql://…` (psycopg); an asyncpg URL is accepted by the ingestor
CHAT_DATABASE_URL	chat agent	direct or session port 5432; the API's URL works, `+asyncpg` is accepted
JWT_SECRET / BULWARK_JWT_SECRET	API / chat agent	identical values; HS256; long and random
CORS_ORIGINS	API, chat server	JSON list including both UI origins
GEMINI_API_KEY	API, ingestor, recommender, chat agent	one key can serve all four
GEMINI_EMBED_MODEL, EMBED_DIM	API, ingestor, recommender, agent	`gemini-embedding-001` and `768` everywhere; it must equal the `vector(768)` column
KAFKA_*	API (optional), ingestor, recommender	same cluster and topic `bulwark.incidents`; the ingestor also writes `bulwark.scans.completed`
KAFKA_GROUP_ID	recommender	`bulwark-solution-recommender`; a new group starts at `KAFKA_AUTO_OFFSET_RESET` (earliest)

Setting	Where	Rule
SIMULATE_EXECUTORS	API and worker	`true` until real executors exist
REQUIRE_CONFIRMATION	chat agent	keep `true`
MCP_URL / BULWARK_API_URL	agent / MCP server	`http://127.0.0.1:8765/mcp` / `http://localhost:8000`
DEFAULT_PASSWORD	API	`admin@123`; the user must change it at first sign-in

Keep `.env` files out of version control and never reuse the values from the delivered `.env.example` files: they contain real-looking credentials and should be rotated.

10. Verify the install

Work down the list; each item needs only the pieces started so far.

- [ ] `curl localhost:8000/health/ready` returns `ready`.
- [ ] Open http://localhost:3000, sign in as `admin` / `admin@123`, choose a new password; the Dashboard opens.
- [ ] The sidebar Incidents badge tooltip reads "of 56 incidents" and Approvals shows 7 incident approvals and 7 solution reviews waiting (sample data).
- [ ] Open an incident in status APPROVED or IN_PROGRESS on the Execution page and run a step; with the worker running, cells move Queued, Running, Succeeded.
- [ ] Vulnerabilities, Solutions, Assets and Users pages list data (Users is visible to ADMIN only).
- [ ] Ingestor: `scan-ingestor --validate-only` reports valid rows for the sample workbook.
- [ ] Recommender: `python -m recommender --incident 12` stores a run and prints similar incidents; the incident's *Similar incident solutions* and *AI Suggested* tabs fill.
- [ ] MCP: `curl localhost:8765/health` is ok; the chat icon in bulwark-ui opens; asking "what needs my attention?" lists incidents.
- [ ] Ask the assistant to change something; it must stop and ask for approval first.

If you want a scripted check, `BULWARK_USER=admin BULWARK_PASSWORD=<your password> npm run smoke` in bulwark-ui runs vulnerability, solution, approval, incident, approval, run end to end against the live API, and `npm test` renders each page.

11. Docker, tests and troubleshooting

Docker. `bulwark-api/docker-compose.yml` starts Postgres 16, a migrate job (`alembic upgrade head`), the API and the worker (copy `.env.example` to `.env` first; Postgres in that compose file has no pgvector, so use a pgvector image if you want AI features). `bulwark-ai/docker-compose.yml` runs the MCP server and chat agent next to an existing API. These compose files were not run in the build sandbox.

Tests.

Project	Command	Note
bulwark-api	`TEST_DATABASE_URL=postgresql+asyncpg://… pytest -q`	drops and recreates the target schema
scan-ingestor, recommender	`TEST_DATABASE_URL=postgresql://… pytest -q`	drops the public schema; recommender needs pgvector
MCP server	`pytest -q`	mocked API, no database
chat agent	`pytest -q`	scripted stand-in for Gemini
both UIs	`npm test`	no backend needed for component tests

Use a scratch database for every test run, never the one holding your data.

Troubleshooting.

Symptom	Cause and fix
`type "vector" does not exist`	pgvector missing; install it and rerun, or enable it in your provider
Runs stay Queued	the worker is not running
Browser shows CORS or cookie errors after login	add the UI's exact origin to `CORS_ORIGINS`; different sites need `REFRESH_COOKIE_SAMESITE=none` plus HTTPS

Symptom	Cause and fix
401 right after login	the password must be changed first; the UI shows the change screen
Account locked	5 wrong passwords; wait 15 minutes or ADMIN uses Unlock on the Users page
`embeddings_unavailable` (503)	`GEMINI_API_KEY` missing or the backfill has not run
Recommender returns `NO_MATCH`	no earlier incidents with approved plans, or `MIN_SIMILARITY` too high
Ingestor exit code 2	bad configuration or Kafka unreachable; check bootstrap address, credentials and `certs/ca.pem`
Chat history not saved	`CHAT_DATABASE_URL` empty, or a transaction pooler is used
Assistant says action not allowed	the API denied it for the signed-in user's roles; this is by design